

计算机系统结构实验报告 Lab01: Flowing_light

uu-matter543

摘要

在 Lab01 中，我成功搭建了 Vivado IDE，学习了如何新建 design 源文件以添加模块、simulation 源文件以添加仿真激励，初步理解了 Verilog 语法、模块设计、仿真方法、调试，收获颇丰。

目录

- 1.实验目的 1
- 2.原理分析 1
 - 2.1 Vivado 工程的基本组成 1
 - 2.2 flowing light 的原理 1
- 3.功能实现 2
 - 3.1 激励文件 flowing_light_tb.v 编写 2
 - 3.2 管脚约束 lab01_xdc.xdc 编写 3
- 4.上板验证 3
- 5.收获与反思 3

1.实验目的

- (1) 通过基础实验熟悉 Xilinx 逻辑设计工具 Vivado 开发环境；
- (2) 了解硬件描述语言 Verilog HDL 描述功能行为的逻辑；
- (3) 通过仿真检验电路设计是否预期；
- (4) 学习使用 I/O Planning 添加管脚约束；
- (5) 实现 flowing light 的功能；
- (6) 熟悉系统硬件开发的基本实验流程。

2.原理分析

2.1 Vivado 工程的基本组成

- (1) design source (.v 文件) (硬件功能设计文件)
- (2) simulation source (.v 文件) (仿真输入激励文件)
- (3) constraints (.xdc 文件) (上板所需管脚约束文件)

2.2 flowing light 的原理

Flowing light 要求在一段时间内，8 个 LED 灯依次轮流亮灭，最后一个 LED 熄灭后，第一个 LED 循环亮起。这个功能可以使用移位来实现控制，实验指导书中也给出了相应的实现。我的代码如下：

```
module flowing_light(
```

```

input clock_p,
input clock_n,
input reset,
output [7:0] led
);
reg [23:0] cnt_reg;
reg [7:0] light_reg;
IBUFGDS IBUFGDS_inst(
    .O(CLK_i),
    .I(clock_p),
    .IB(clock_n)
);
always @ (posedge CLK_i) begin
    if (!reset) cnt_reg <= 0;
    else cnt_reg <= cnt_reg + 1;
end
always @ (posedge CLK_i) begin
    if (!reset) light_reg <= 8'h01;
    else if (cnt_reg == 24'hfffffff) begin
        if (light_reg == 8'b10000000) light_reg <= 8'b00000001;
        else light_reg <= light_reg << 1;
    end
end
end
assign led = light_reg;

```

定义端口 clock_p 和 clock_n 用于实现差分时钟，reset 用于复位，cnt_reg 用于存储流水的时间间隔，light_reg 即为 8 位流水 LED 的控制信号。

reset=0 时复位为初始状态，reset=1 时，cnt_reg 在每个时钟上升沿+1，并在每次到达 24 位全 1 时更新 light_reg 状态，即 LED 每过 2^{24} 个时钟周期进行一次移位。两个 always 模块分别实现流水间隔计数和 LED 信号更新，最后用 assign 连接 led 和 light_reg，保证流水灯控制信号的实时更新。

3.功能实现

本实验比较简单，基于上述的原理容易实现 flowing light 的功能。在实现 flowing_light.v 后，编写 flowing_light_tb.v 激励文件用以仿真测试，编写 lab01_xdc.xdc 管脚约束用以上板。

3.1 激励文件 flowing_light_tb.v 编写

```

reg clock_p;
reg clock_n;
reg reset;
wire [7:0] led;
flowing_light example_flowving_light(
    .clock_p(clock_p),
    .clock_n(clock_n),
    .reset(reset),
    .led(led)
)

```

